Lumina AI – Final Project Report

AI-Powered Multi-Agent Retrieval-Augmented Generation (RAG) System

> Items marked **[Record]** are blanks to be filled with real measurements from the running application. Do not report any capability as implemented, or any test as passed, until it has been observed.

1. Abstract

Lumina AI is a multi-agent knowledge retrieval system that combines Retrieval-Augmented Generation (RAG), semantic search and natural-language processing to answer questions from uploaded documents. Documents in PDF, DOCX, TXT and CSV formats are extracted, cleaned, chunked, embedded and indexed. Five specialized agents (Query Understanding, Retrieval, Response Generation, Clarification and Conversation Memory) are coordinated by an orchestrator to produce source-grounded answers, ask for clarification when a question is ambiguous, and handle follow-up questions. The platform adds voice interaction, response transparency and an analytics module that records real query activity to expose low-confidence answers, unanswered questions and knowledge gaps. It is built with a React/Vite frontend, a Python FastAPI backend and the Google Gemini API, and is evaluated across AI/NLP, Remote Sensing/Earth Observation and Cybersecurity knowledge domains. **[Record: add one sentence summarizing the headline test results once measured.]**

2. Introduction and Problem Statement

Organizations, students and professionals keep knowledge in PDFs, Word files, text files and spreadsheets. Locating a specific fact by hand is slow. Keyword search misses answers phrased with different words, and general-purpose chatbots may produce answers that are not supported by the user's own documents, without showing where they came from. They also lose context across follow-up questions, and unanswered questions are rarely tracked.

A system is therefore needed that understands natural-language questions, retrieves semantically relevant content, answers only from evidence, remembers conversational context, shows its sources, and reveals where the knowledge base is incomplete.

3. Existing System and Its Limitations

- Manual searching takes time.

- Keyword search may miss semantically related information.
- Large documents are difficult to navigate.
- Unsupported AI-generated answers reduce reliability.
- Sources may not be clearly identified.
- Follow-up questions force users to repeat context.
- Unanswered questions are not systematically tracked.

4. Objectives

1. Build an AI-powered knowledge retrieval and question-answering platform.
2. Support PDF, DOCX, TXT and CSV uploads.
3. Extract, clean and chunk content for retrieval.
4. Generate embeddings and support semantic similarity search.
5. Classify queries as factual, procedural, comparative or ambiguous.
6. Retrieve relevant chunks using a configurable Top-K process with a relevance threshold.
7. Generate answers grounded in retrieved information.
8. Provide source attribution and retrieval transparency.
9. Support clarification and follow-up questions through conversation memory.
10. Provide voice input and text-to-speech.
11. Record real query activity and identify unanswered questions and knowledge gaps.
12. Test and optimize the system using actual results.

5. Background

- **Retrieval-Augmented Generation** supplies a language model with retrieved passages at answer time, reducing reliance on parametric memory and letting answers reflect private or current documents.
- **Semantic search** compares embedding vectors, so a question can match a passage that uses different wording.
- **Chunking** trades context against precision: large chunks keep context but dilute relevance, small chunks are precise but may lose meaning.
- **Multi-agent decomposition** splits a complex task into specialized steps that are easier to test and improve individually.

Add formal citations here according to your institution's reporting style.

6. System Design

The design has seven parts: user interface, API layer, ingestion module, storage (vector store and metadata database), orchestration layer with five agents, analytics layer, and the external Gemini API. Full diagrams, data models and API contracts are in the companion document *Lumina AI – Technical Documentation*.

Decision	Rationale
RAG instead of fine-tuning	Knowledge is updated by uploading documents, with no retraining
Separate agents with structured outputs	Each stage can be tested, logged and tuned independently
Relevance threshold plus low-confidence threshold	Distinguishes "no evidence" from "weak evidence"
One clarification round per query	Avoids interrogation loops
Memory kept apart from the knowledge index	Prevents chat history contaminating document retrieval
Sources taken from the actual retrieval result	Keeps transparency truthful
Analytics from stored logs only	Ensures the dashboard reflects real use

6.1 Multi-agent summary

Agent	Responsibility
Query Understanding	Classify query type, detect follow-ups, rewrite to a standalone query
Retrieval	Embed query, retrieve Top-K chunks, filter by threshold, keep metadata
Response Generation	Produce an answer only from retrieved evidence, report sources and confidence
Clarification	Ask one focused question when the query is ambiguous
Conversation Memory	Track recent turns, topics and entities; handle topic switches

6.2 Core workflows

- **Ingestion:** upload, validate, extract, clean, chunk, embed, index, update knowledge base.
- **Query resolution:** user input, memory context, classification, (clarification), retrieval, relevance check, grounded generation, memory update, analytics logging, response with sources.

7. Implementation

Frontend. React with Vite. Pages: AI Assistant (chat, sources panel, confidence badge, voice controls), Knowledge Base (upload, list, status, delete), Analytics (metrics and gap lists) and Settings.

Backend. Python FastAPI exposing `/api/health`, `/api/documents`, `/api/upload`, `/api/query`, `/api/analytics` and `/api/settings`.

Ingestion. Validation, format-specific text extraction, cleaning, overlapping chunking, batch embedding, vector indexing and metadata storage.

Agents and orchestration. Prompted agents with JSON-structured outputs, coordinated by an orchestrator that branches between clarification, no-evidence response and grounded answer.

Voice. Web Speech API for recognition and speech synthesis, with permission and error handling.

Analytics. Every query is logged; metrics and gap lists are computed from the log.

7.1 Implementation status

Complete from the running application, using only the values Implemented, Partial or Not implemented, and attach evidence such as a screenshot or log entry.

Capability	Status	Evidence
Document upload and indexing (4 formats)	[Record]	
Query classification	[Record]	
Semantic retrieval with Top-K and threshold	[Record]	
Grounded answer generation	[Record]	
Source attribution and transparency panel	[Record]	
Clarification flow	[Record]	

Capability	Status	Evidence
Conversation memory	[Record]	
Voice input and text-to-speech	[Record]	
Analytics from real queries	[Record]	
Knowledge-gap detection	[Record]	

8. Testing and Validation

Each test is recorded as **Passed, Failed, Warning or Not Tested** from actual execution.

8.1 Test plan

ID	Area	Test	Expected result	Result
T1	Upload	Upload one PDF, DOCX, TXT and CSV file	All processed, listed as ready, chunk counts shown	[Record]
T2	Upload	Upload unsupported, empty and oversized files	Clear rejection message	[Record]
T3	Retrieval	Ask 5 questions per domain with known answers	Correct chunk in Top-K	[Record]
T4	Classification	Factual, procedural, comparative, ambiguous samples	Correct category	[Record]
T5	Grounding	Check answers against retrieved chunks	No unsupported claims	[Record]
T6	No evidence	Ask about a topic absent from the KB	"Not found in documents" response, logged as unanswered	[Record]
T7	Clarification	Ask "What about memory?"	One focused clarifying question; clarified query then answered	[Record]
T8	Memory	Follow-up using pronouns, then switch topic	Follow-up resolved; no context bleed after switch	[Record]

ID	Area	Test	Expected result	Result
T9	Transparency	Compare displayed sources with logged retrieval	Exact match	[Record]
T10	Voice	Recognize speech, edit, submit, play TTS, deny mic permission	Works; errors handled	[Record]
T11	Analytics	Run a known set of queries, compare dashboard to log	Counts match	[Record]
T12	Gap detection	Repeat unanswered questions on one topic	Topic appears as a gap	[Record]
T13	Performance	Measure latency per stage and ingestion time	Recorded, bottlenecks identified	[Record]
T14	Robustness	Model API failure, backend down	Friendly error, no crash	[Record]

8.2 Multi-domain evaluation

Milestone 4 requires testing across three knowledge domains. Load suitable documents for each and record results separately.

Domain	Example documents	Sample questions	Result
AI / NLP	RAG, transformer and embeddings papers or notes	"What is RAG?" "How does chunk size affect retrieval?"	[Record]
Remote Sensing / Earth Observation	Notes on SAR, NDVI, soil moisture products	"What does NDVI measure?" "How does SAR backscatter relate to vegetation?"	[Record]
Cybersecurity	Material on encryption, OWASP, incident response	"How do symmetric and asymmetric encryption differ?" "What are the steps of incident response?"	[Record]

Do not claim all three domains were validated unless documents were loaded and results are recorded.

8.3 Suggested metrics

- **Hit rate@K:** share of test questions where a chunk containing the answer appears in the Top-K.
- **MRR:** mean reciprocal rank of the first correct chunk.
- **Groundedness:** share of answers whose claims are supported by retrieved chunks (manual review of a sample).
- **No-evidence accuracy:** share of out-of-scope questions correctly declined.
- **Classification accuracy:** correct query types over a labelled test set.
- **Latency:** median and 95th percentile for total and per-stage time.

9. Optimization

Optimization should be driven by the measurements above.

Area	Possible change	Measure
Chunking	Tune size and overlap; split on paragraph boundaries	Hit rate@K
Retrieval	Adjust Top-K and threshold using score distributions of correct vs incorrect hits	Hit rate, no-evidence accuracy
Latency	Batch embeddings, cache query embeddings, avoid reprocessing unchanged files, reduce prompt size, run independent steps concurrently	Median latency
Prompts	Tighten grounding instructions, enforce JSON schema	Groundedness, parse failures
Clarification	Calibrate trigger so it fires only for genuinely ambiguous queries	Clarification precision
Reliability	Timeouts, retries with backoff, graceful fallback messages	Error rate

Before/after table to complete:

Change	Metric	Before	After
[Record]			

10. Results

Fill this section with actual figures only.

- Documents loaded: [Record] across [Record] domains; total chunks: [Record].
- Retrieval hit rate@K: [Record]; MRR: [Record].
- Groundedness on reviewed sample: [Record].
- Out-of-scope questions correctly declined: [Record].
- Median response time: [Record] ms; 95th percentile: [Record] ms.
- Voice: tested in [Record browser]; outcome [Record].
- Analytics: total queries logged [Record]; unanswered [Record]; low-confidence [Record]; knowledge gaps found [Record].

11. Expected Benefits

- Faster access to information in uploaded documents.
- Natural-language interaction with the knowledge base.
- Semantic retrieval beyond exact keyword matching.
- Clearer, evidence-based answers.
- Better handling of follow-up questions.
- Greater visibility into retrieval quality.
- Identification of missing knowledge through analytics.
- More accessible interaction through voice.

Actual performance improvements must be established through testing rather than assumed.

12. Limitations

- Scanned or image-only PDFs need OCR, and poor extraction harms retrieval.
- Tables and complex layouts may be flattened and lose structure.
- Embedding quality bounds semantic search quality.
- Irrelevant retrieved chunks can degrade answers.

- Relevance and confidence scores are not guarantees of factual correctness.
- Dependence on an external model API introduces latency, quota and availability risk.
- Voice features depend on browser support and microphone permission.
- The system cannot answer questions whose knowledge is absent from the document set.
- Large files need more processing time and memory.
- Limitations observed during testing should be added here. **[Record]**

13. Future Enhancements

- OCR and table-aware extraction for scanned documents.
- Hybrid retrieval (keyword plus vector) and re-ranking.
- Scaling to larger knowledge bases with a production vector database.
- Multilingual queries and documents.
- Automated groundedness and answer-quality evaluation.
- Caching and incremental re-indexing.
- Role-based access control and per-user knowledge bases.
- Richer gap analysis, with suggested documents to add and feedback buttons for flagging bad answers.
- Improved voice accessibility and multilingual speech.

14. Conclusion

Lumina AI demonstrates how RAG and a multi-agent architecture can turn a collection of documents into a conversational, evidence-based knowledge assistant. Source transparency makes answers checkable, and analytics make missing knowledge visible so the knowledge base can be improved over time. The strength of the final claims depends on the evidence gathered during testing, which should be recorded in Sections 7 to 10.